Crocodile Write-up

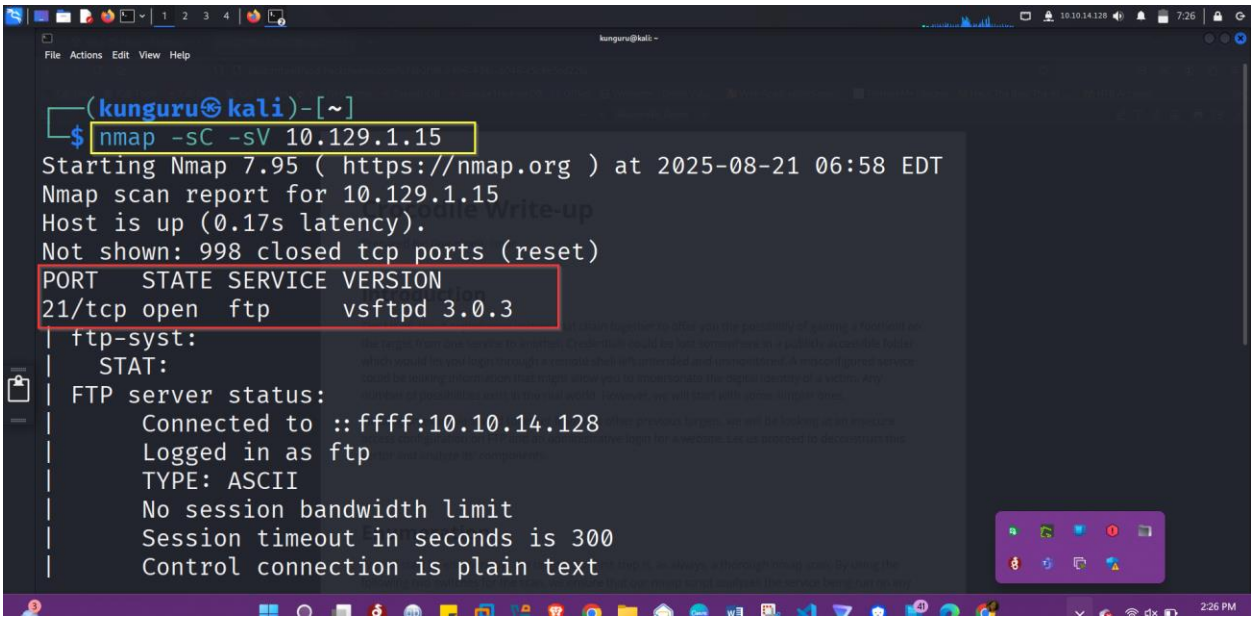
Intruduction

Tools

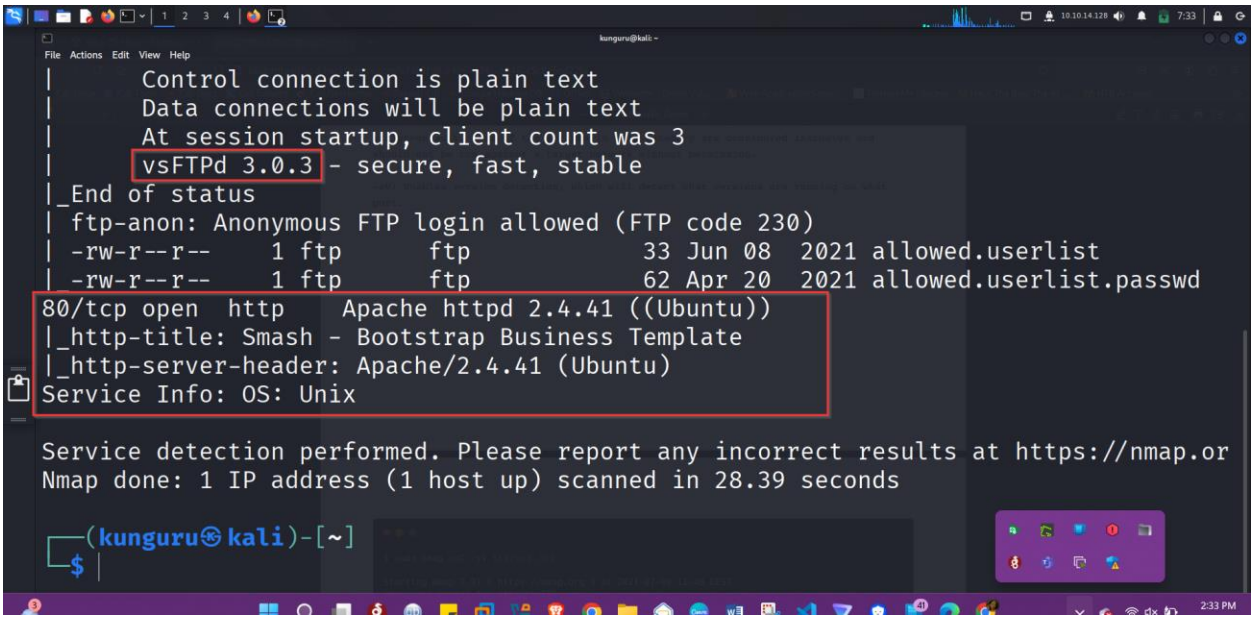
- Kali Linux
- Nmap
- Ftp cli
- Firefox
- Gobuster
- Wappalyzer

Enumeration

We will run an nmap scan of our target.



- -sC: Performs a script scan using the default set of scripts.
- -sV: Enables version detection, which will detect what versions are running on what port.



The nmap scan:

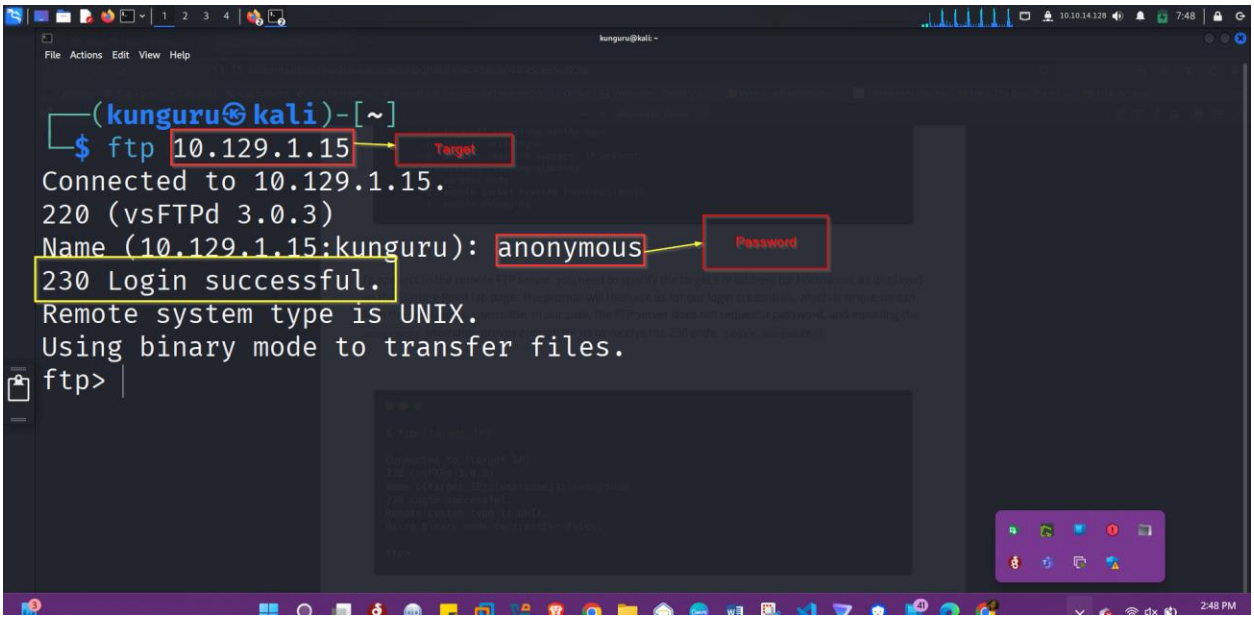
I. Identified that port 21/tcp was open and running an FTP (File Transfer Protocol) service.

✓ This indicates the system is accepting connections for file transfers, which could potentially allow anonymous or credentialed access depending on its configuration.

II. revealed port 80/tcp open, hosting an HTTP service powered by Apache httpd 2.4.41 on Ubuntu.

✓ The server header identified as Apache/2.4.41 (Ubuntu), and the HTTP title revealed the page served was Smash - Bootstrap Business Template.

✓ This suggests the host is running a web service that may be susceptible to version-specific vulnerabilities or misconfigurations.



To connect to the remote FTP server, you need to specify the target's IP address.

The prompt will then ask us for our login credentials, which is where we can fill in the anonymous username.

In our case, the FTP server does not request a password, and inputting the anonymous username proves enough for us to receive the 230 code, Login successful.



With the `dir` command, we can check the contents of our current directory on the remote host, where two interesting files catch out attention.

They seem to be files left over from the configuration of another service on the host, most likely the HTTPD Web Server.

Their names are descriptive, hinting towards a possible username list and associated passwords.

```
ftp> get allowed.userlist
local: allowed.userlist remote: allowed.userlist
229 Entering Extended Passive Mode (|||45207|)
150 Opening BINARY mode data connection for allowed.userlist (33 bytes).
100% |***************************|    33       23.33 KiB/s    00:00 ETA
226 Transfer complete.
33 bytes received in 00:00 (0.23 KiB/s)
```

```
ftp> get allowed.userlist.passwd
local: allowed.userlist.passwd remote: allowed.userlist.passwd
229 Entering Extended Passive Mode (|||40573|)
150 Opening BINARY mode data connection for allowed.userlist.passwd (62 b
ytes).
100% |***************************|    62       19.22 KiB/s    00:00 ETA
226 Transfer complete.
62 bytes received in 00:00 (0.43 KiB/s)
```

Both files can easily be downloaded using the `get` command.

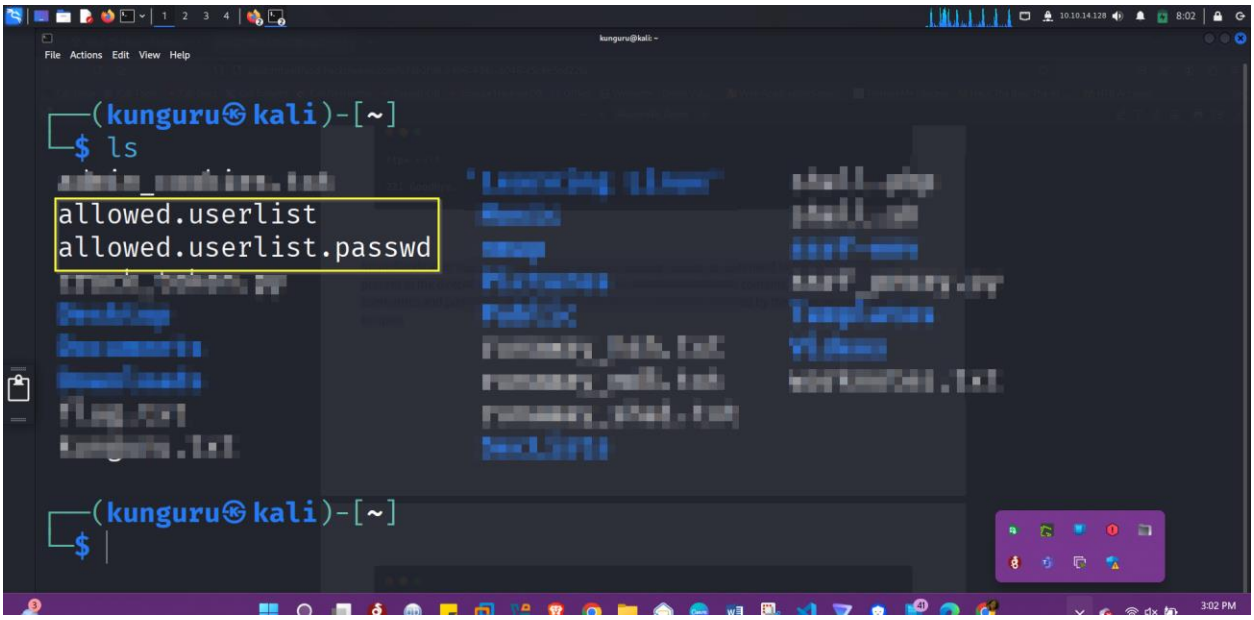
The FTP service will report the download status completion back to you during this phase.

```
ftp> exit
221 Goodbye.
```

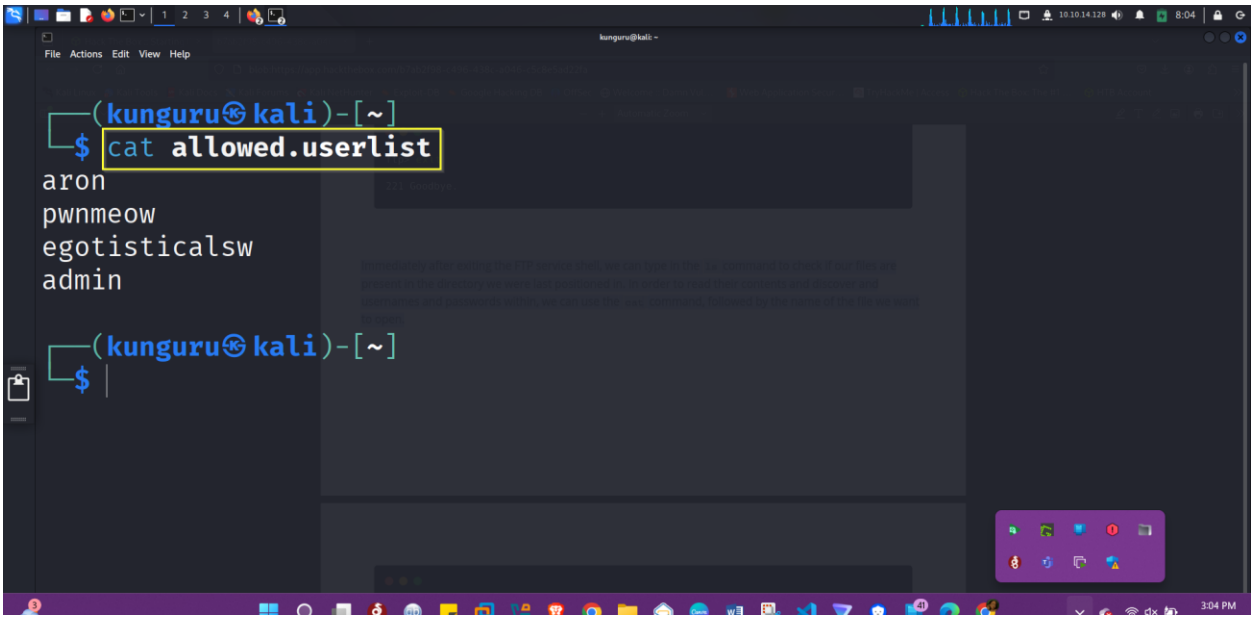
Termination of the FTP connection can be done by using the exit command.

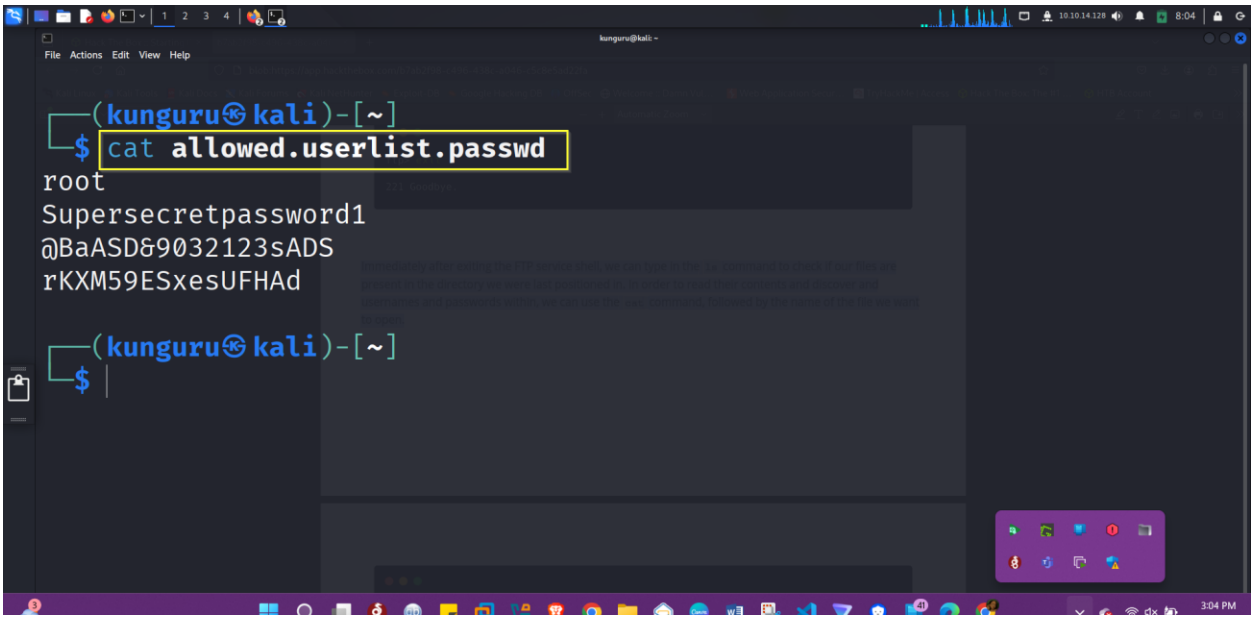
This will return the current terminal tab to its' previous state.

Immediately after exiting the FTP service shell, we can type in the `ls` command to check if our files are present in the directory we were last positioned in.



To read their contents and discover and usernames and passwords within, we can use the `cat` command, followed by the name of the file we want to open.



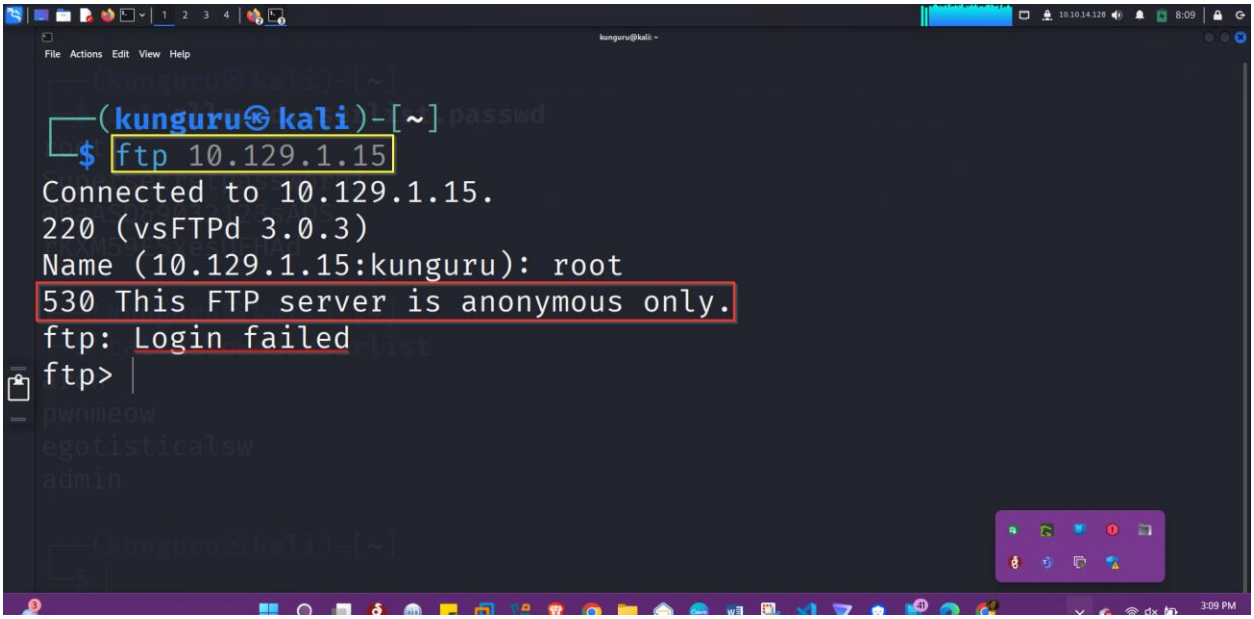


Foothold

After the credentials have been obtained, the next step is to check if they are used on the FTP service for elevated access or the webserver running on port 80 discovered during the nmap scan.

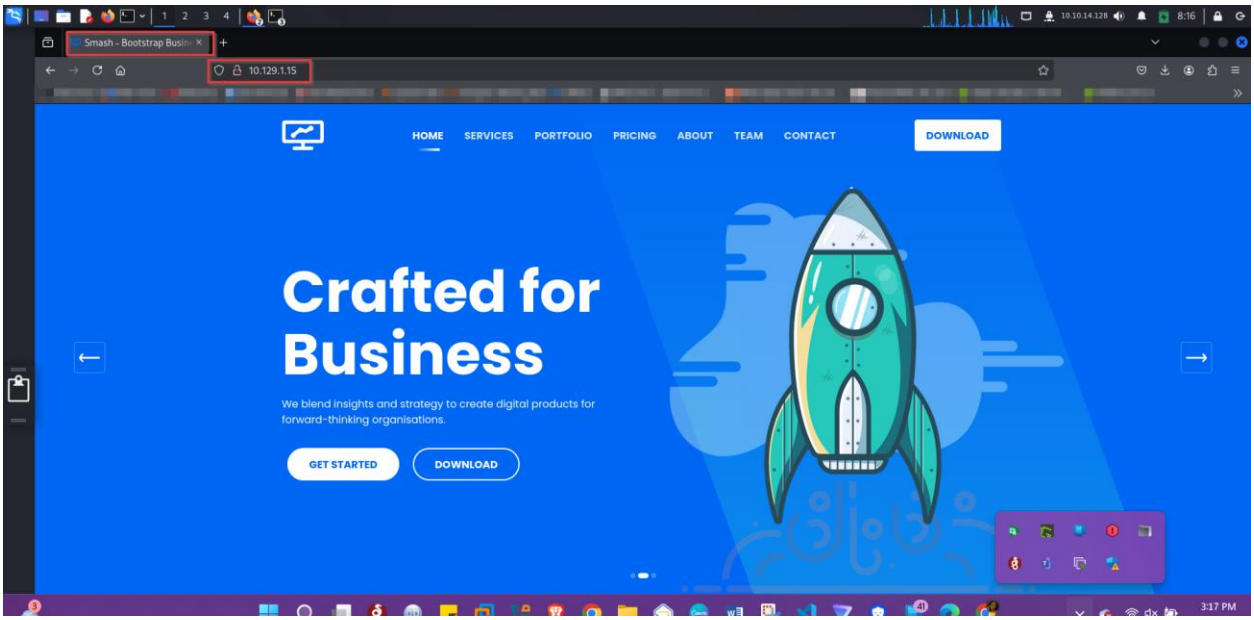
Attempting to log in with any of the credentials on the FTP server returns error code 530 This FTP server is anonymous only.

No luck here, so we can exit the FTP service shell.



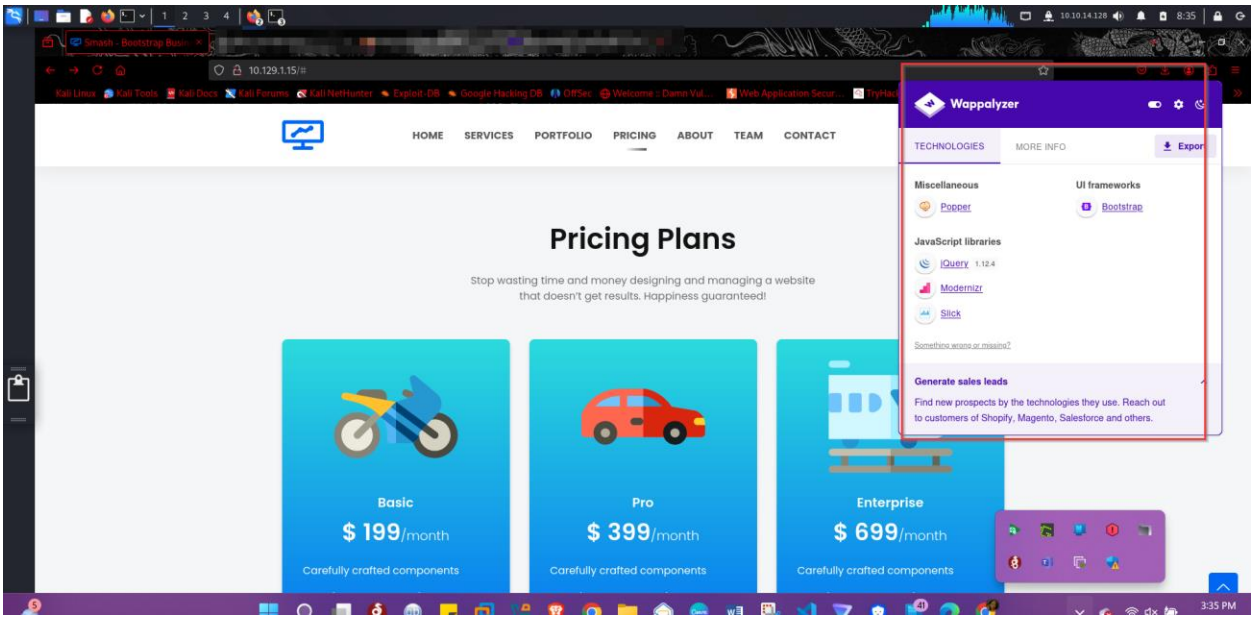
During the nmap scan, the service running on port 80 was reported as Apache httpd 2.4.41, an Apache HTTP server.

Typing in the IP address of the target into our browser's URL search bar results in this webpage. It seems to be a storefront for a server hosting company.



To gain more insight into the technology they have used to create their website and possibly find any associated vulnerabilities, we can use a handy browser plug-in called Wappalyzer.

This plug-in analyzes the web page's code and returns all the different technologies used to build it, such as the webserver type, JavaScript libraries, programming languages, and more.



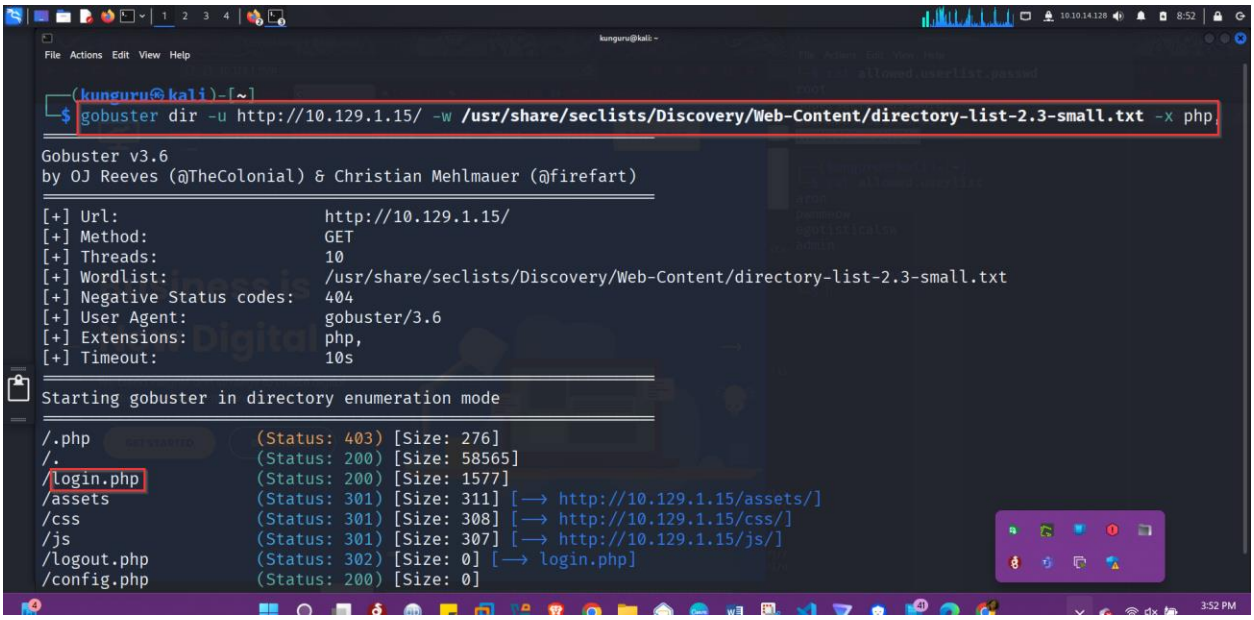
From the output of Wappalyzer, we can note some of the more interesting items, specifically the PHP programming language used to build the web page. However, nothing gives us a direct plan of attack for now.

Meanwhile, navigating around the page using the tabs and buttons provided on it leads us nowhere.

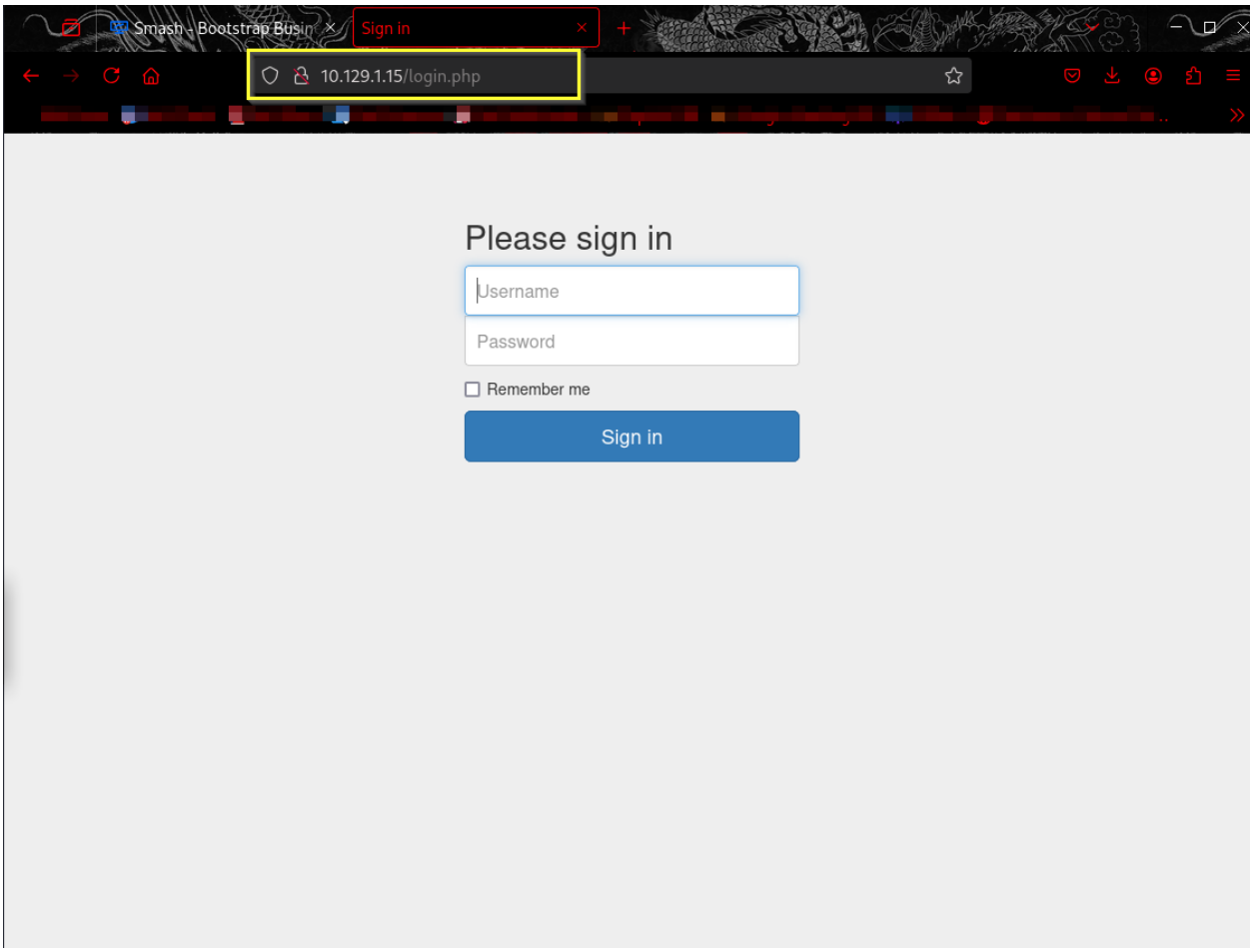
We shall go through dir busting. Using gobuster as our tool of choice, we can use the following switches for the script to get the fastest, most accurate results.

```
dir : Uses directory/file enumeration mode.
--url : The target URL.
--wordlist : Path to the wordlist.
-x : File extension(s) to search for.
```

For the -x switch, we can specify php and html to filter out all the unnecessary clutter that does not interest us. PHP and HTML files will most commonly be pages. We might get lucky and find an administrative panel login page that could help us find leverage against the target in combination with the credentials we extracted from the FTP server.

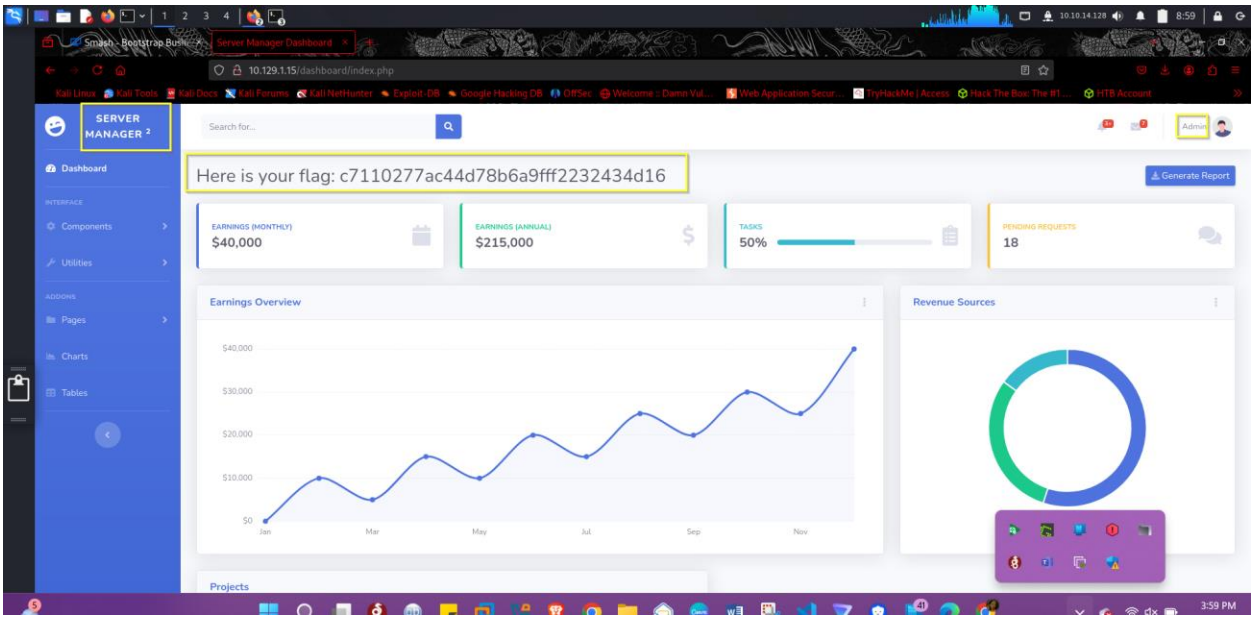


One of the most interesting files gobuster retrieved is the /login.php page. Navigating manually to the URL, in the form of http://{10.129.1.15}/login.php, we are met with a login page asking for a username/password combination



After attempting several username/password combinations, we manage to log in and are met with a Server Manager admin panel.

Once here, an attacker could manipulate the website in whichever way they desired, causing havoc for the userbase and owners or extract more information that would assist them with gaining a foothold on the servers hosting the web page.



We successfully got the flag! It is displayed for us at the top of the admin panel.

Flag: c7110277ac44d78b6a9fff2232434d16